

Continuum - Evidence-First Engineering Operations Control Plane

Deterministic Runs + Audit-Ready Proof for High-Stakes Financial Systems

What Continuum is

Continuum turns high-stakes engineering work into deterministic, replayable **Runs** that automatically generate audit-ready evidence and enforce policy gates.

The problem (what banks feel daily)

Critical changes and incidents rely on tribal orchestration across Jira, Git, CI, Postman/Newman, logs, databases, and environment rituals. The result: slow cycle time, inconsistent verification, and manual proof.

The insight

The missing primitive is a **Run**: the unit of execution that captures inputs -> plan -> execution -> verification -> evidence -> publishable report. Banks don't just ship code; they must **prove** it.

How it works (end-to-end)

Ingest (ticket/workflow/CI/alert) -> **Plan** (deterministic step graph) -> **Execute** (plugins) -> **Verify** (structured checks) -> **Prove** (tamper-evident evidence bundle) -> **Publish** (report + Jira/CI).

What Continuum outputs (the receipt)

Every run produces a consistent evidence bundle: scenario snapshot, per-step inputs/outputs, logs and checks, hashed manifest, summary with pass/fail and root-cause details, plus a shareable HTML report.

Why financial companies buy it

Continuum compresses cycle time while reducing risk: standardized verification, policy gates for releases, repeatable incident playbooks, and audit readiness by default.

Differentiation

CI/CD runs pipelines. Continuum runs **reality**: cross-repo workflows, flaky shared environments, approvals, evidence requirements, deterministic resume/retry/selective execution, and policy enforcement.

Wedge -> platform

Start with payments-grade workflows (FedNow/RTP regressions, return-of-funds, end-to-end verification). Expand to fraud/risk, identity/KYC/AML, core banking integration, and regulated change control across the org.

CTA: Replace manual orchestration with provable runs. One command. One receipt. Zero ambiguity.